

Nota de referência

Este material reúne minhas anotações de estudo desenvolvidas ao longo do curso de Visão Computacional da NexVisual, realizado em parceria com o Instituto Eldorado.

O conteúdo aqui apresentado foi organizado com fins educacionais e de revisão pessoal, a partir das aulas, explicações, atividades práticas e exemplos trabalhados durante o curso. As anotações refletem meu processo de aprendizagem, podendo conter adaptações, complementações, interpretações próprias e exemplos desenvolvidos durante os estudos.

Agradeço à NexVisual e ao Instituto Eldorado pela oportunidade de aprendizado e pela contribuição para minha formação na área de Visão Computacional.

Aula 1 - Detecção e extração de características

A detecção de objetos é quando queremos encontrar onde está um elemento específico e não apenas reconhecer o que ele é. A detecção de objetos tem duas etapas:

- Identificar o objeto de interesse;
- Localiza-lo dentro da imagem.

Os objetos podem estar em posições diferentes, com iluminação variada ou parcialmente escondidos. Uma das abordagens para extrair características dessas imagens e efetivamente detectar os objetos apesar de inúmeros cenários e condições variáveis é identificar padrões únicos que representem um objeto. Esses padrões são como uma impressão digital de uma imagem e precisam ser invariáveis.

Extratores:

Os extratores são algoritmos utilizados para analisar uma imagem e identificar informações relevantes nela. No contexto de visão computacional, eles podem detectar pontos importantes da imagem e também calcular características associadas a esses pontos.

Exemplos de extratores de características:

Como vimos no módulo 1, o HoG (Histogram of Gradients) é uma técnica para detecção de objeto onde são calculados os gradientes da imagem e a distribuição desses gradientes é usada para detecção. Essas informações ajudam o algoritmo a reconhecer formas e contornos. HoG é mais conhecido como um descritor de características. Ele extrai informações da imagem a partir da distribuição dos gradientes, ou seja, das direções das bordas e contornos. Ele não trabalha exatamente com “keypoints” como SIFT e ORB; normalmente ele descreve

regiões ou janelas da imagem para ajudar em tarefas como detecção de pessoas e objetos.

Outra técnica famosa é o SIFT (Scale-Invariant Feature Transform) por ser invariante a alteração de escalas. O SIFT está presente dentro da OpenCV. SIFT é um detector e descritor de características locais. Ele detecta keypoints, que são pontos de interesse na imagem, e depois calcula um descritor para cada ponto. A grande vantagem dele é ser robusto a mudanças de escala, rotação e parcialmente a variações de iluminação.

O ORB vêm de Oriented FAST and Rotated BRIEF, sendo uma alternativa aberta e eficiente que combina rapidez e boa precisão de desempenho quando usado em cenário de detecção em tempo real, como por exemplo em aplicativos de celular (App's) ou câmeras embarcadas. ORB também é um detector e descritor de características locais.

Pontos importantes da parte prática:

Keypoints:

Os keypoints, ou pontos de interesse, são pontos específicos da imagem considerados importantes por possuírem características marcantes, como cantos, bordas, mudanças de textura ou regiões com alto contraste. Esses pontos são úteis porque tendem a ser reconhecíveis mesmo quando a imagem sofre alterações, como rotação, mudança de escala ou variação de iluminação.

Descritores de keypoint:

Os descritores de keypoint são representações numéricas das características ao redor de cada ponto de interesse. Eles funcionam como uma espécie de “assinatura” do keypoint, descrevendo a região próxima a ele por meio de valores que representam padrões de intensidade, textura e direção das bordas. Esses descritores permitem comparar keypoints entre imagens diferentes, sendo muito usados em tarefas como reconhecimento de objetos, alinhamento de imagens e criação de panoramas.

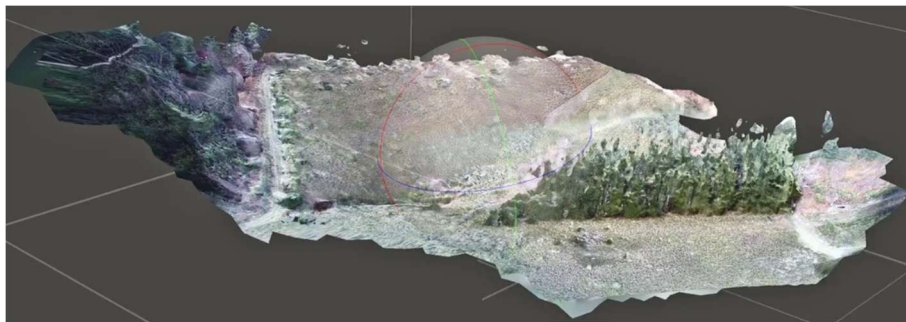
Aula 2 - Correspondências de características

Baseia-se em comparar características em comum entre diferentes imagens e encontrar correspondências. Para tanto podemos usar os algoritmos de matching. Eles analisam os descritores de uma imagem, e procuram os mesmos padrões em outra imagem. Quando há correspondências sabemos que o objeto está presente na imagem que estamos analisando. Além da detecção de objetos a análise de correspondências permite alinhar duas imagens diferentes sendo esta última aplicação usada para:

- Foto panorâmica, onde várias imagens são tiradas em sequência, alinhadas e unidas;



- No uso de realidade aumentada o sistema precisa reconhecer superfícies e objetos reais, para posicionar elementos virtuais corretamente;
- Veículos autônomos usam visão computacional para identificar placas e sinais de trânsito em tempo real;
- Sistema de busca por imagens como o Google Lens necessitam do matching para encontrar imagens semelhantes ou reconhecer produtos;
- Na indústria robôs usam esse tipo de técnica para inspecionar e organizar peças em linhas de produção;
- Usado em reconstrução de topografias tridimensionais com uso de imagens;



- Robôs também usam a técnica de matching para se movimentar e localizar em ambientes.

Técnicas de matching

Algoritmo de força bruta promove a comparação de todos os pontos uns com os outros.

K-Nearest Neighbours, usa aprendizado de máquina para encontrar as correspondências mais próximas.

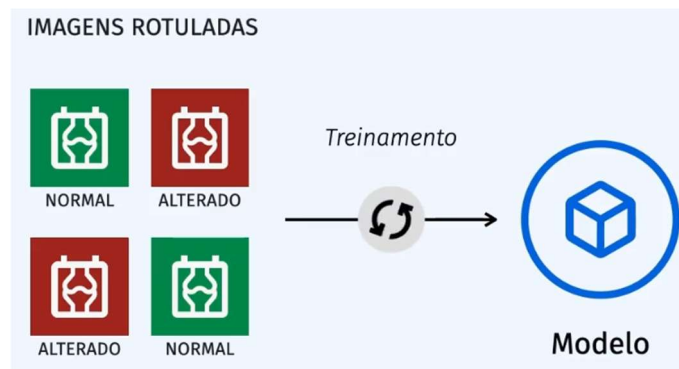
FLANN, uma biblioteca eficiente quando se tem grandes volumes de dados.

Aula 3 - Classificadores supervisionados

Classificação de imagens é um problema clássico de aprendizagem de máquina. Dado uma imagem, o sistema deve atribuir um rótulo a ela, por exemplo:

- Um modelo que analisa exames médicos e classifica entre exame normal ou exame com alteração;
- Classificação de animais: Gato ou cachorro;

Para tanto o modelo deve ser treinado previamente com um conjunto de imagens já rotuladas (já se sabe a qual classe pertencem).



Durante o treinamento o modelo aprende padrões e características que diferenciam uma classe da outra e ao receber uma nova imagem tenta prever o rótulo mais provável.

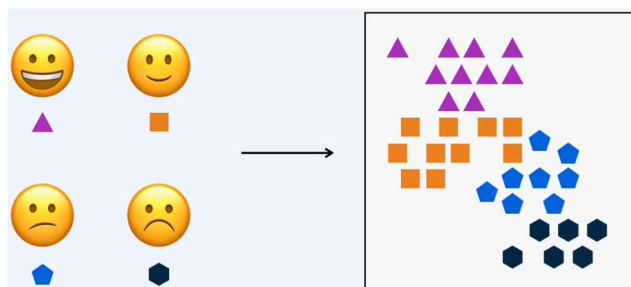
Modelos de aprendizado de máquina clássico

Existem vários tipos de classificadores supervisionados, sendo os mais famosos:

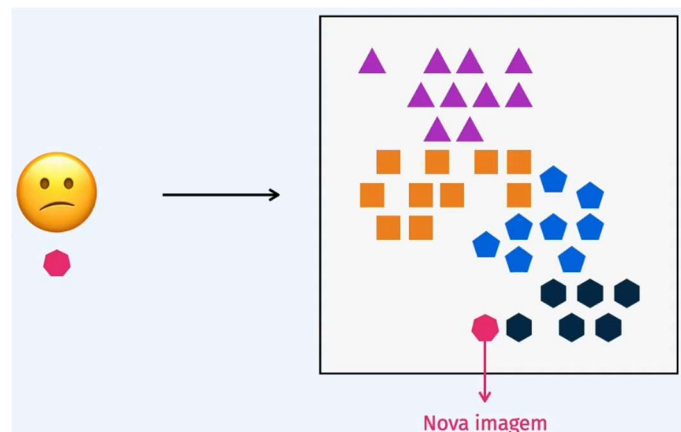
KNN – K-Nearest Beighbors:

O KNN é o modelo mais intuitivo, ele analisa a distância entre uma nova imagem e as imagens do conjunto de treinamento (geralmente essa distância é a euclidiana que é aquela que mede o quão perto ou longe dois pontos estão).

No KNN primeiro extraímos as características das imagens, depois colocamos essas características em um espaço de representação.

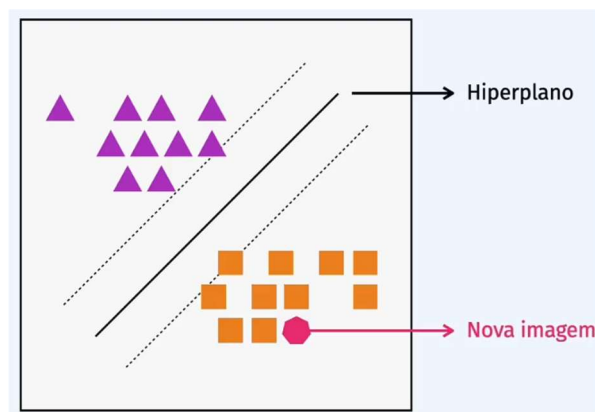


Quando chega uma nova imagem, o KNN procura os k vizinhos mais próximos e faz uma votação. A classe com mais votos é a escolhida. Essa votação pode ser feita de maneira simples, apenas contando quais as classes dos vizinhos ou usando outra métrica de estatística.



SVM – Support Vector Machines:

O SVM assim como o KNN precisa de uma base rotulada e das características extraídas. Mas ao invés de medir distância entre vizinhos, o SVM cria fronteiras, chamadas de hiperplanos, que delimitam as regiões das classes no espaço de representação. Os hiperplanos são posicionados de forma a maximizar a separação entre as classes, garantido que o modelo saiba de que lado cada nova imagem deve ficar.

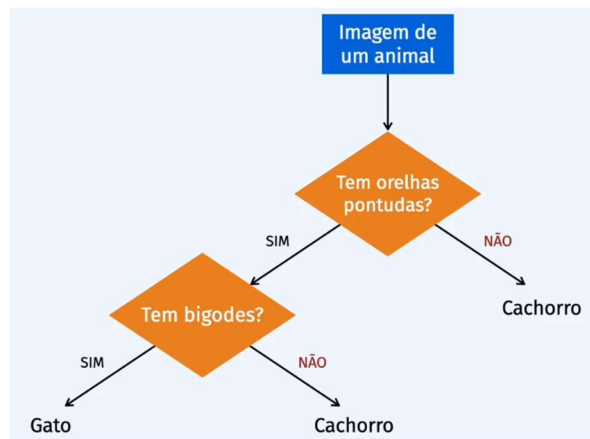


Árvores de decisão:

Algoritmo de máquina supervisionada e necessita da base de dados rotulada. Útil quando precisamos entender por que o modelo tomou determinada decisão. Com a base rotulada o algoritmo cria uma árvore que com base nos dados decide qual classificação pertencente.

Redes Convolucionais Profundas:

São do tipo data-intensive e precisam de grandes quantidades de dados para aprender. Em 2012 fizeram um desafio com um dataset massivo para o problema de classificação de imagem (ImageNet). Para a solução um grupo propôs a AlexNet, uma rede que revolucionou os resultados da área.



Aula 4 - Divisão de dados

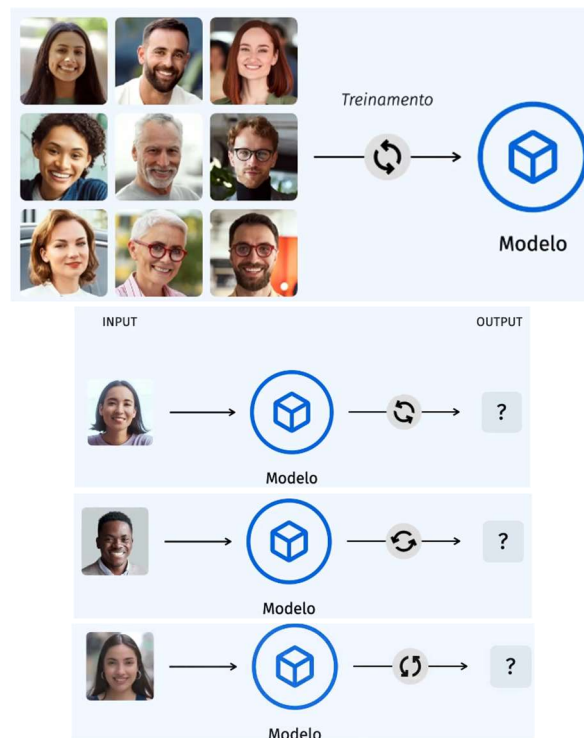
Os dados são essenciais para o desempenho de um modelo, mas não basta ter muitos dados eles precisam ser bons, representativos e confiáveis. A base de dado precisa representar bem o problema. Por exemplo se o objetivo do código for detectar leões em uma imagem não faz sentido incluir fotos em ambientes com neve (onde não poderão existir leões). O modelo deve aprender a reconhecer leão no contexto correto, em savana seu habitat natural.



Viéses nos dados

Se algum tipo de dado é subamostrado(ou seja, aparece em quantidade menor do que os outros dados durante o treinamento) isso será refletido no modelo.

Imagine um modelo de detecção de pessoas treinado apenas com a imagem de europeus e norte-americanos. Esse modelo terá dificuldade em identificar pessoas asiáticas, africanos ou latino-americanas.



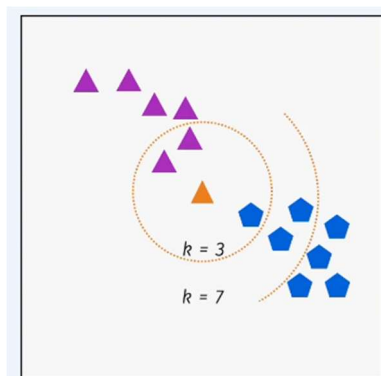
Ao preparar os dados sempre devemos analisar se:

- Os dados são representativos considerando a problemática;
- Há algum viés que precisa ser corrigido.

Divisão de dados

Normalmente temos uma quantidade limitada de dados e além de treinar o modelo, precisamos validar se os resultados obtidos são bons, além de verificar quais conjuntos de parâmetros geram os melhores resultados. Então depois de garantir uma boa base de dados o próximo passo é dividir esses dados.

No algoritmo KNN por exemplo, se alterarmos o valor de K temos diferentes modelos que terão diferentes performances.

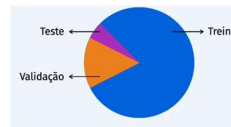


Então precisamos usar parte dos dados para treinar, parte para validar e outra para testar o modelo. Portanto em geral fazemos três divisões principais:

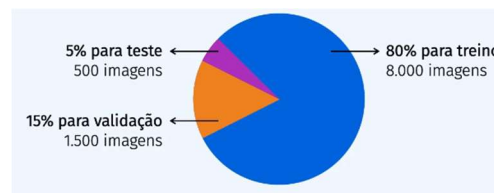
Treino: Onde o modelo aprende a partir de exemplos.

Validação: Usada para validar os parâmetros e garantir que o aprendizado esteja indo bem. Assim sendo possível treinar, ajustar e avaliar o modelo com segurança;

Teste: Usado para avaliar do desempenho final, com dados que o modelo nunca viu para testar sua performance simulando a utilização do modelo no mundo real.



Imagine que você tem 10.000,00 imagens de pessoas para um modelo de detecção de pessoas. Uma boa divisão seria:

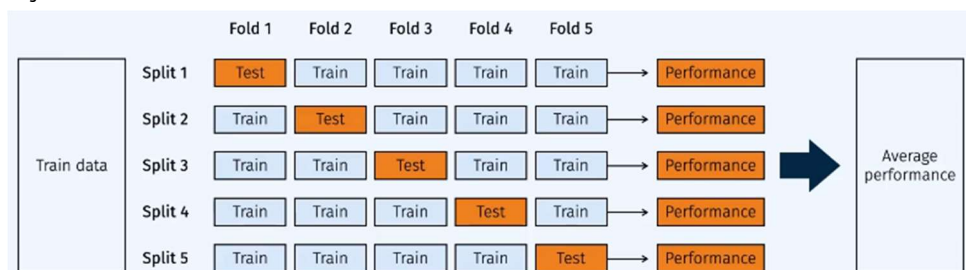


Desta forma, o modelo aprende é ajustado e testado em etapas diferentes. Um ponto de atenção é que não pode haver contaminação de dados. Todo o conjunto, treino, validação e o teste são disjuntos, isso significa que não pode haver exemplos que se repetem em qualquer um dos conjuntos.

Validação cruzada

Mesmo assim, ainda podem ser introduzidos vieses que podem comprometer o resultado. Uma forma de reduzir esse risco é usando a técnica chamada validação cruzada. Essa técnica não é tão usada em visão computacional e principalmente em modelos de aprendizado profundo, mas é importante para trazer mais relevância estatísticas aos dados.

Na validação cruzada dividimos o conjunto em partes iguais, as chamadas folds e a cada rodada, uma parte diferente é usada para a validação, por exemplo, em uma validação cruzada com 5 folds, com K igual a 5, o modelo é treinado cinco vezes e em cada vez usamos uma parte diferente dos dados para validar. No final fazemos a média dos resultados. Isso dá mais confiabilidade estatística e evita que o modelo pareça bom apenas por sorte em uma divisão específica. Esse processo é mais demorado porque o modelo precisa ser treinado várias vezes, mas em compensação a análise fica muito mais sólida.

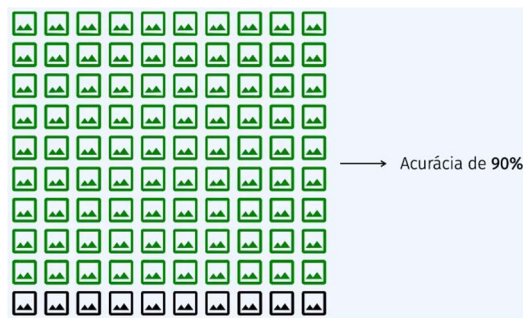


Aula 5 - Métricas de desempenho

Métricas de desempenho são usadas para determinar se o modelo criado irá funcionar bem no mundo real. São métricas definidas matematicamente que ajudam a medir de forma objetiva o desempenho do modelo. Cada métrica olha para um aspecto diferente da performance. Então cada uma tem suas vantagens e desvantagens e capturam aspectos diferentes do modelo.

Vamos iniciar falando de métricas de problemas de classificação. A primeira métrica é a:

Acurácia: Ela mostra quantas previsões o modelo acertou em relação ao total. Por exemplo: Se o modelo acerta 90 de 100 imagens a acurácia é de 90%.



Mas a acurácia pode enganar, principalmente em situações desbalanceadas. Por exemplo, considera um classificador que analisa exames médicos e procura por anomalias nas imagens. Se 95% dos exames são normais e o modelo sempre responde normal a acurácia será de 95% mas 0% de utilidade. Por isso precisamos de métricas mais detalhadas. Para entender melhor o desempenho usamos duas métricas que se complementam:

Precisão: Responde à pergunta: Das previsões que o modelo faz, quantas estavam realmente corretas? Ex: A precisão mostra quantas pacientes dignosticados como doentes realmente estão doentes.

Recall (Ou revocação): Dos casos realmente positivos, quantos o modelo conseguiu encontrar? Ex: Quantos doentes reais o modelo conseguiu identificar.

F1 Score: Combina precisão e recall usando uma média harmônica gerando um número único que mostra o equilíbrio entre elas. Ex: Modelo de detecção de pedestres, se o modelo identifica postes ou sombras como pessoas ele comete falsos positivos e a precisão cai. Mas se ele não detecta pedestres reais, o recall cai. O F1 score ajuda a perceber esse balanço;

Exemplo completo:

Um modelo detectou 100 objetos como pedestres, mas só 60 era, pessoas reais. Precisão, 60%. Na imagem havia 80 pedestres reais, dos quais 60 foram detectados. Recall, 75%. Logo o F1 Score será de:

$$F1 = 2 \times \frac{60 \times 75}{60 + 75} = 66,66\%$$

Ou seja, o modelo deve ser melhorado!

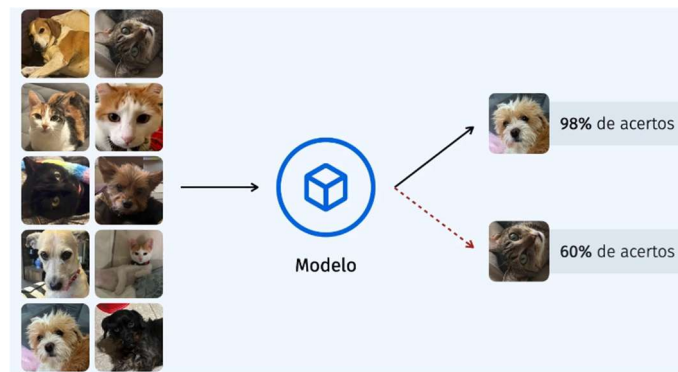
	POSITIVO	NEGATIVO
POSITIVO	Positivo Verdadeiro (ou PV)	Falso Positivo (ou FP)
NEGATIVO	Falso Negativo (ou FN)	Negativo Verdadeiro (ou NV)

$$Acurácia = \frac{PV + NV}{PV + NV + FN + FP}$$

Aula 6 - Visualização de resultados

A visualização de resultados consiste em analisar as métricas de uma modelo e verificar a qualidade do desempenho. Embora as métricas sejam fundamentais visualizar os resultados pode revelar padrões, erros sistemáticos e oportunidades de melhoria que os números sozinhos não mostram.

Por exemplo, a figura abaixo mostra um modelo de classificação de animais com várias classes. O modelo pode ter acurácia alta (98%), mas erra com frequência uma classe específica (Classificação de gato, 60%). Analisando apenas a acurácia do modelo e seu valor final poderia se concluir erroneamente que o modelo estava adequado devido a acurácia geral alta por esse motivo a visualização de resultados ajuda a identificar esse tipo de falha.



Matriz de confusão:

É uma forma de visualizar a performance de modelos de classificação com múltiplas classes de um jeito simples.

	CACHORRO	GATO	PEIXE	
CACHORRO	✓	×	×	✓
GATO	×	✓	×	Modelo
PEIXE	×	×	✓	

Interpretando a matriz da imagem, as linhas representam as classes verdadeiras dos dados e as colunas mostram as classes que o modelo previu. O que esperamos encontrar são valores altos na diagonal principal. Isso significa que o modelo acertou aquela classe na maior parte das vezes. Se alguma classe tiver desempenho ruim o valor na diagonal vai aparecer baixo.

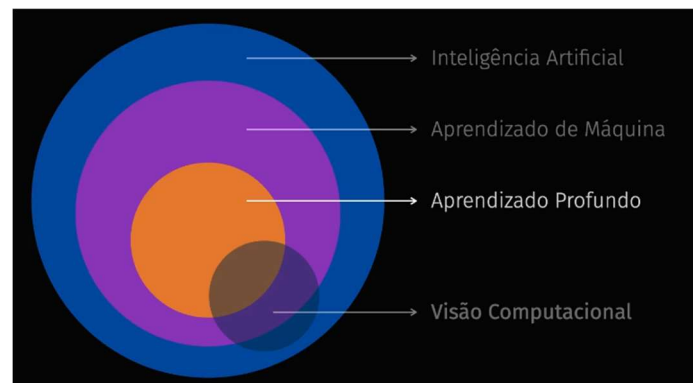
Curvas de perda

Outra forma de analisar erros em modelos de aprendizagem de máquinas é observando as curvas de perda ou curvas de loss. Quando treinamos um modelo, estamos realizando um processo de otimização. Normalmente, existe uma função matemática que guia esse processo, seja minimizando ou maximizando algum valor. Durante o treinamento, podemos acompanhar como esse valor evolui ao longo do tempo. Essas curvas são muito utilizadas em modelo de aprendizado profundo. O treino costuma ser mais demorado e acompanhar esse comportamento ajuda a saber se estamos de fato evoluindo o modelo. Da mesma maneira, podemos gerar curvas para as métricas de avaliação. Assim, acompanhamos a performance durante o treinamento. Isso também permite parar

o treinamento antes do previsto, quando percebemos que a melhora já não compensa o tempo de processamento gasto. Outro ponto essencial é analisar essas curvas tanto no conjunto de treino quanto no de validação, isso ajuda a identificar overfit, ou seja, quando o modelo está apenas decorando os dados de treino se aprender padrões reais.

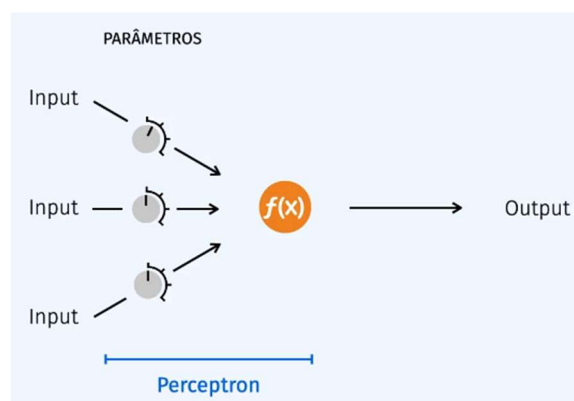
Aula 7 - Introdução a redes neurais

Aprendizado profundo é uma subárea do aprendizado de máquina, mas com um foco especial: Redes neurais.

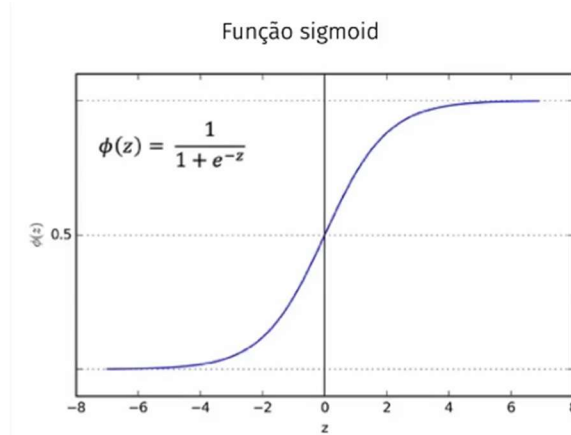


O aprendizado profundo foi desenvolvido com uma tentativa ousada de simular como um cérebro humano funciona. O cérebro tem bilhões de neurônios conectados por sinapses, eles recebem informações, processam e enviam sinais para outros neurônios. Essa estrutura inspirou os cientistas a criar modelos computacionais que funcionam de forma semelhante. Unidades chamadas neurônios artificiais recebem entradas, aplicam uma função matemática e geram uma saída.

Em 1958, surgiu o modelo mais famoso, o perceptron. Um único neurônio artificial com múltiplas entradas aplica pesos a essas entradas, soma os resultados e passa por uma função de ativação que decide se aquele neurônio artificial está ativo ou não.



Uma função muito usada é a sigmoide que independente da entrada retorna um valor entre 0 e 1.

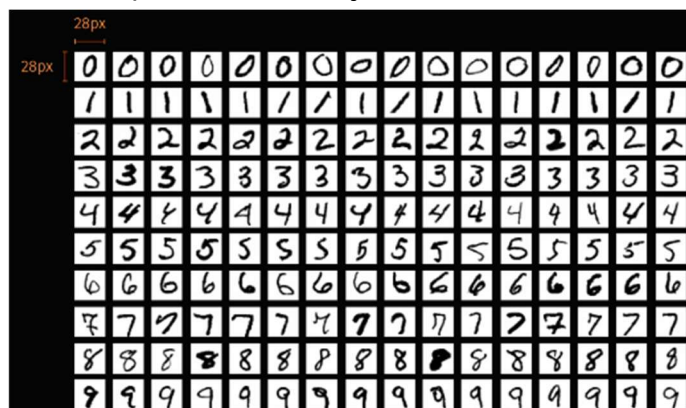


Durante muito tempo esses elementos básicos constituíram as redes neurais artificiais. Com isso, redes com perceptrons já eram capazes de aproximar qualquer função. Essa propriedade se chama aproximador universal.

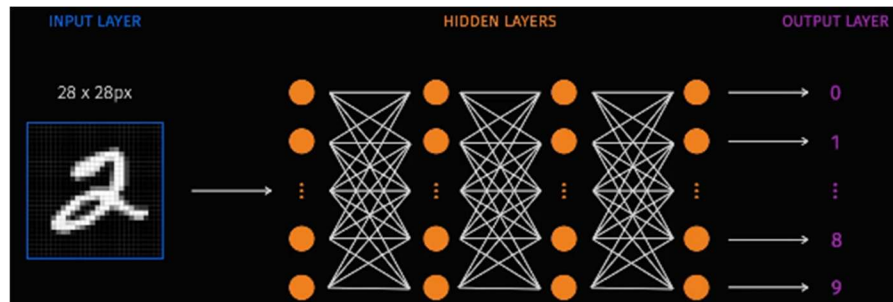
Aula 8 - Introdução à visão com aprendizado profundo

Modelos de aprendizado profundo são *data-intensive*, ou seja, precisam de muitos dados para aprender bem. E foi justamente a criação de grandes conjuntos de dados que impulsionou o avanço de aprendizado profundo em tarefas de visão computacional. O uso de redes neurais é uma grande quantidade de dados possibilitou uma mudança de paradigma para o aprendizado de máquina. Ao invés de utilizar métodos para extrair características das imagens e utilizar essas informações para treinar o modelo, os próprios modelos agora ficam encarregados de extrair as características. Assim, o modelo pode, durante o treinamento escolher quais características deve extrair para resolver a tarefa.

Agora iremos fazer dos três *datasets* mais históricos, sendo o primeiro o MNIST – Modified National Institute of Standards and Technology ou Instituto Nacional de Padrões e Tecnologia Modificado. O MNIST é uma base de dados formada por números escritos à mão, contendo 70 mil imagens em tons de cinza e cada uma delas tem 28 X 28 pixels de resolução.



Em cada imagem tem um dígito entre zero a nove que é escrito à mão. A tarefa do dataset é de classificar as imagens, definindo qual o número presente na imagem.

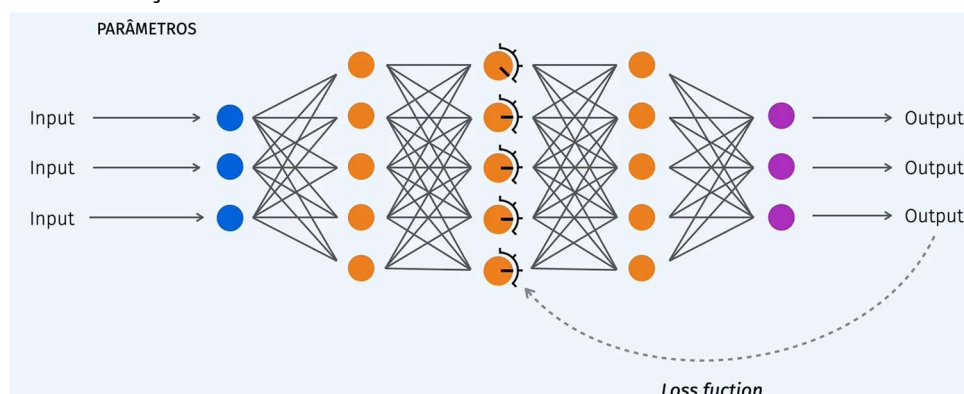


Outra base de dados muito importante foi a CIFAR – Canadian Institute for Advanced Research ou Instituto Canadense de Pesquisa Avançada. Esse dataset possui uma variação com 10 classes e com 100 classes. A versão de 10 classes do CIFAR contém 60 mil imagens coloridas, sendo que cada uma possui resolução de 32 x 32 pixels, sendo essas imagens de objetos ou animais e a tarefa também é classificar a imagem nas possíveis classes.

O último conjunto de dados e talvez o marco da revolução é o ImageNET, o conjunto de dados que popularizou as redes convolucionais em tarefas de visão computacional para classificação de imagens. Essa base reúne mais de 1,3 milhões de imagens organizadas em 1 mil classes diferentes. Essas imagens não possuem padrão de resolução e foram tiradas em ambientes reais na natureza, o que torna o desafio ainda mais interessante.

Em 2012, usando essa base de dados que as redes convolucionais ganharam, destaque. Foi nesse momento que elas se popularizaram e deram início a era moderna do Deep Learning. Além dos dados três outros fatores foram fundamentais para o avanço tecnológico o desenvolvimento do Algoritmo de Backpropagations, o surgimento de ativação ReLU e as GPUs.

Algoritmo de Backpropagations: Atualiza os pesos das camadas, as redes neurais usando uma função de erro chamada loss functions.



Ativação ReLU (Rectfied Linear Unir ou Unidade linear retificada):

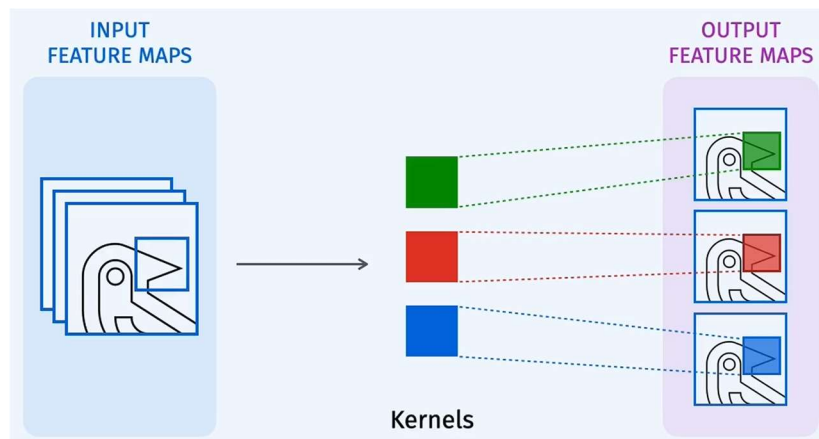
Permitiu uma computação mais simples que a sigmoide anteriormente utilizada e que simplificava os modelos e ainda garantia a adição de não linearidades ao modelo.

GPUs (Graphics Processong Unit: Todos esses avanços tecnológicos só foram possíveis por conta das GPUs, hardwares modernos de paralelização, que são processadores matriciais e todos os cálculos dessas redes neurais podem ser feitos por meio de computação entre matrizes. Eles permitiram que modelos maiores pudessem ser treinados em velocidade aceitável, superando métodos antigos.

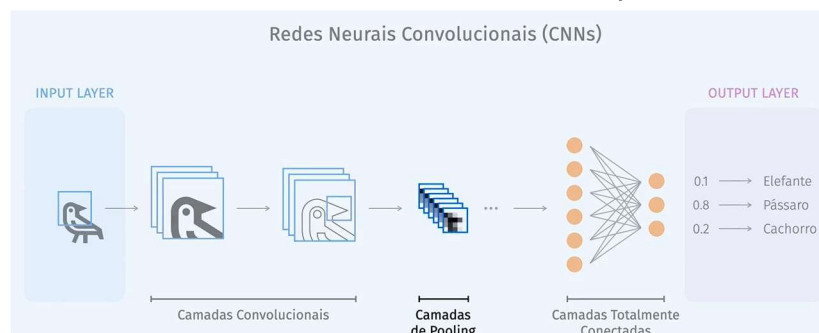
Aula 9 - Redes neurais convolucionais (CNNs)

Foram projetadas para trabalhar com dados visuais, conseguindo analisar a estrutura espacial das imagens, capturando padrões locais como bordas, texturas e formas. As CNN's funcionam basicamente com três camadas:

Camadas convolucionais: Aplicam filtros ou kernels sobre a imagem. Esses filtros são aprendidos durante o treinamento e cada um detecta um padrão específico. Por exemplo, um filtro pode detectar bordas horizontais enquanto outro detecta bordas verticais ou até mesmo texturas específicas.



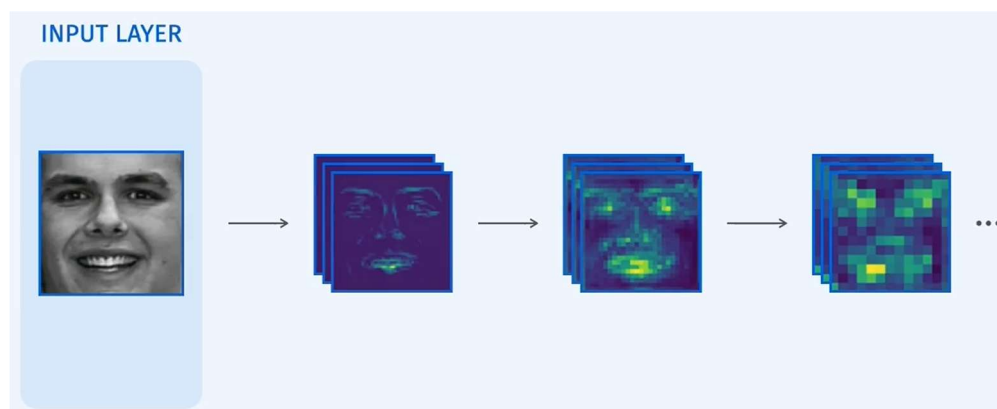
Camadas de Pooling: Ajudam a reduzir a dimensão dos dados, imagens tem muita informação, uma foto full HD tem mais de 2 milhões de pixels.



O pooling reduz a dimensão deixando o modelo mais leve, reduzindo o número de parâmetros e mantendo apenas o que importa para o reconhecimento, como se fosse um resumo inteligente da imagem.



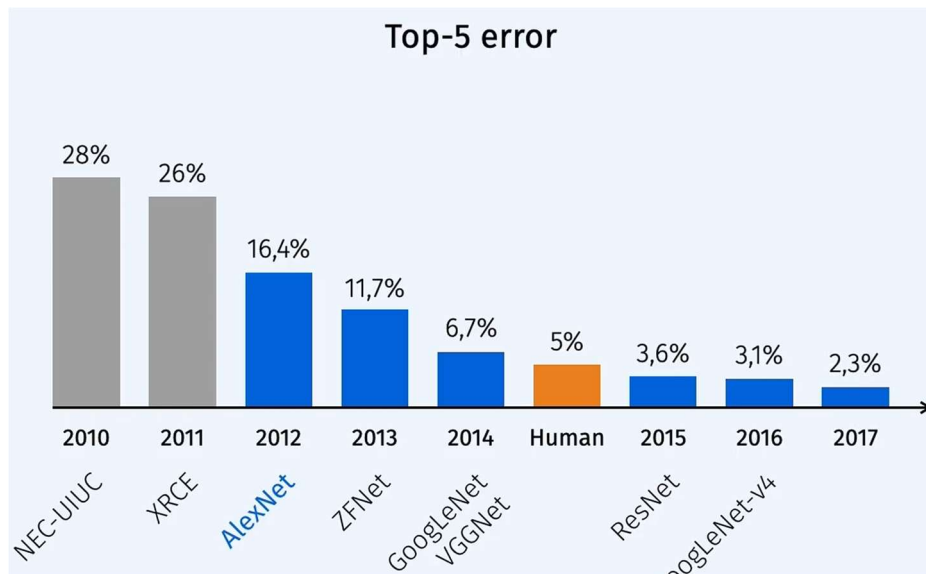
Camadas Totalmente conectadas: Funcionam como um classificador MLP (Múltiplos classificadores), as camadas convolucionais e o pooling geram representações vetoriais das imagens que permitem ao MLP uma performance de classificação superior, uma vez que as camadas de pooling e convolucionais apreciam a relação espacial entre os pixels



Modelos de CNNs famosos:

Lenet 5: Desenvolvida nos anos 90 por Ian Lecun, ganhador do prêmio Turing pelos avanços em aprendizado profundo, foi uma das primeiras CNNs aplicadas com sucesso, especialmente em reconhecimento de dígitos manuscritos utilizando a base de dados MNIST.

AlexNet: Apresentada em 2012, foi um divisor de água. Vencedor do desafio ImageNet com uma margem impressionante, mostrando que redes convolucionais profundas podiam superar todos os métodos tradicionais, AlexNet usava ReLU e foi treinada com GPUs.



Aula 10 - Modelos pré-treinados

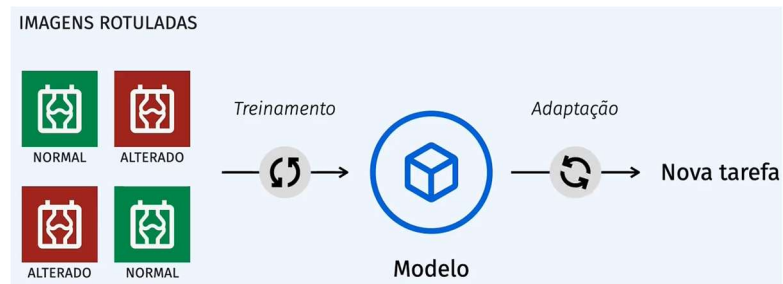
A facilitar a compreensão da estrutura de uma rede neural convolucional, utilizamos diagramas de arquitetura que representam de forma visual a sequência de camadas que compõem da rede desde a entrada, até a saída:



Modelos pré-treinados

Treinar uma CNN do zero pode ser bem caro, pois exigem muitos dados, muito tempo de treino, além da dependência das GPUs. A comunidade científica encontrou uma solução, o uso de modelos pré-treinados. Modelos treinados anteriormente em grandes bases de dados como a imagenet e aprenderam a reconhecer uma enorme variedade de padrões visuais. Esse conhecimento é reaproveitado sendo necessário

apenas a adaptação do modelo para uma nova tarefa. Sendo essa técnica chamada de Transferlearning.



Dois modelos pré treinados muito conhecidos são:

VGG: Recebeu esse nome por ter sido criada pelo Visual Geometry Group da Universidade de Oxford, criada para a tarefa de classificação de imagens e ficou conhecida por ter uma arquitetura profunda, porém bastante padronizada, com blocos de convolução pequenos e empilhados sobre os outros. Os modelos VGG16 e VGG19 com 16 e 19 camadas, são os mais conhecidos e utilizados até hoje como base para diversas aplicações.

ResNet: O nome vem de rede Residual Network, essas camadas funcionam criando pontes entre partes da arquitetura. As camadas mais profundas recebem informações diretamente de camadas menos profundas, sem necessariamente passar por todas as outras camadas.